Day 2 Labs: Build a Business Dashboard

Total Duration: 4 labs × 30-45 min | **Difficulty:** Beginner → Intermediate
Prerequisites: Claude Code installed, authenticated, Node.js 18+, Git installed

---

Overview

Across four labs today, you'll build **one complete application from scratch** — a Business Dashboard with real-time metrics, task management, and a SQLite database. Each lab builds on the last, so by the end of the day you'll have a professional app you built entirely with Claude Code.

No SQL server required. Everything runs locally with SQLite — zero configuration.

What you'll build across all four labs: - A REST API with 10+ endpoints - A SQLite database with automatic seeding - A live web UI showing metrics and tasks - Data visualization with charts and filters - CSV export functionality - Git version control with clean conventional commits - CLAUDE.md as your project's policy engine - Production-quality error handling and security

---

Pre-Flight Check (5 min)

Before Lab A, verify your environment. Open a terminal and run each command:

> ***Which terminal?*** *Use whatever terminal you normally use — PowerShell, Windows Terminal, Mac Terminal, or Linux shell. Claude Code works in all of them.*

1. Check Node.js

```
node -v
```

Expected output: `v18.x.x` or higher

2. Check Git

```
git --version
```

Expected output: `git version 2.x.x` (any recent version)

3. Check Claude Code

```
claude --version
```

Expected output: A version number like `1.x.x`

> ***Not installed?*** *See the Installation slide or ask your instructor.*

□Pre-Flight Complete

- [ ] Node.js 18+ installed
- [ ] Git installed
- [ ] Claude Code installed

That's it. Claude handles everything else from here.

Lab A: The Magic Moment (30-45 min)

After: Installation & Agentic Loop slides

Goal

Go from an empty folder to a running web app in a single prompt. This is the moment that changes how you think about building software.

Steps

1. Create Your Project and Start Claude

```
mkdir business-dashboard
cd business-dashboard
claude
```

> ***Windows note:*** `mkdir` *works in all terminals (CMD, PowerShell, Mac, Linux).*

You're now in an interactive Claude Code session. **From this point forward, everything happens inside Claude.**

2. The Build Prompt

Copy and paste this prompt exactly:

```
Initialize this as a git repo and build me a business dashboard web application:

Tech stack: Node.js, Express, SQLite (via better-sqlite3), vanilla HTML/CSS/JS frontend
No external CSS frameworks — write clean, modern CSS

Database tables:
1. metrics — id, name, value (number), category, recorded_at
2. tasks — id, title, status (pending/in-progress/completed), priority (high/medium/
low), assigned_to, created_at, completed_at

API endpoints:
- GET /api/metrics (filter by ?category=)
- POST /api/metrics
- GET /api/tasks (filter by ?status= and ?priority=)
- POST /api/tasks
- PUT /api/tasks/:id
- DELETE /api/tasks/:id
- GET /api/dashboard (summary: all metrics + task counts by status + high priority task
s)

Frontend: Dark theme dashboard at / showing all metrics as cards and tasks as a list.
Seed the database with 8 sample metrics across categories (financial, customers, operat
ions, hr) and 5 sample tasks.

Initialize npm, install dependencies, create everything, and start the server. Use
port 3100 (or PORT env var).
Include a .gitignore for node_modules and *.db files.
```

Watch Claude work through the agentic loop: 1. **Plan** — Analyzes requirements, decides on file structure 2. **Execute** — Creates files, runs `npm install`, writes code 3. **Validate** — Checks for errors, tests the server

> ***Instructor Note:*** *This is the "magic moment." Students see Claude build a complete, working application from a single prompt. Let it run — don't interrupt.*

3. See It in the Browser

Open your browser to `http://localhost:3100`

You should see a dark-themed dashboard with 8 metric cards and 5 tasks.

4. Test the API Through Claude

Ask Claude to test its own work:

```
Test the API: add a new metric called "New Leads This Week" with value 42
in the "sales" category. Then add a task "Review Q2 projections" with
high priority assigned to me. Then complete task 1.
Show me the results after each operation.
```

Refresh the dashboard in your browser. The new data should appear.

5. Your First Git Commit

```
Commit everything with the message "feat: initial business dashboard with API and UI"
```

> ***You may see a Skills prompt.*** *Claude Code has built-in Skills that auto-detect common tasks like committing. If you see a permission prompt asking to use a "commit" skill, select* ***Yes*** *— this is normal. It means Claude is using a workflow skill to handle the commit. You'll learn more about Skills in Day 3.*

What Just Happened?

This is the **agentic loop** in action:

1. **Planning:** Claude read your prompt, understood the requirements, and created a mental plan
2. **Execution:** It autonomously created files, ran npm commands, wrote code
3. **Validation:** It checked that the structure made sense before presenting it to you

You didn't tell Claude *how* to build it. You told it *what* you wanted. Claude: - Knew to use Express (industry standard for Node.js APIs) - Chose better-sqlite3 (fast, zero-config database) - Structured the code properly (server setup, routes, database layer) - Added error handling without being asked - Seeded test data so the dashboard isn't empty - Built a complete frontend with dark theme styling

This is fundamentally different from autocomplete or code snippets. Claude understood the **intent** and delivered a **working system**.

Vague vs. Specific Prompting

Let's see the difference. Try this vague prompt first:

```
Add validation.
```

Look at what Claude does — it guesses which routes, which fields, what error format.

Now try the specific version:

```
In @src/index.js, modify only the POST /api/tasks route. Validate:
- title: must be non-empty string, max 200 characters
- priority: must be one of "high", "medium", "low"
- assigned_to: optional, but if provided must be non-empty string
Return 400 with { error: "description of what's wrong" } for invalid requests.
```

Compare the outputs. **The specific prompt produces exactly what you want on the first try.** The vague prompt requires 2-3 rounds of correction.

> ***Teaching point:*** *Specificity isn't extra work — it's less work. One precise prompt beats three vague ones.*

Optional: Go Deeper (if you finish early)

Add more endpoints:

```
Add these endpoints to the API:
- GET /api/metrics/summary — returns min, max, average value per category
- GET /api/tasks/overdue — returns tasks that are pending and older than 7 days
- PATCH /api/tasks/:id/assign — changes the assigned_to field only
Test each one and show me the results.
```

Explore the code Claude wrote:

```
Show me the file structure of this project and explain each file's purpose
```

Try breaking something on purpose:

```
What happens if I POST a task with no title? Show me the error response.
```

□Lab A Checkpoint

You should now have: - [ ] A running Express server on port 3100 - [ ] SQLite database with metrics and tasks - [ ] Working web UI at http://localhost:3100 - [ ] Input validation on POST /api/tasks - [ ] 7+ API endpoints all working - [ ] First git commit

□Return to slides when your instructor is ready.

Lab B: Your Project Brain (30-45 min)

After: CLAUDE.md & Memory Architecture slides

Goal

Teach Claude your team's conventions so every future change follows your standards automatically — without you asking.

Key insight: CLAUDE.md is policy, not documentation. It doesn't describe your project — it controls Claude's behavior.

Steps

1. Clear Context

In your Claude Code session:

```
/clear
```

This starts a fresh context window. The project files are still on disk — Claude just starts with a clean conversation.

2. Create CLAUDE.md

Tell Claude:

```
Create a CLAUDE.md file in the project root with these conventions:

# Business Dashboard - Project Conventions

## Tech Stack
- Node.js + Express backend
- SQLite via better-sqlite3 (no external DB required)
- Vanilla HTML/CSS/JS frontend (no frameworks)
- Dark theme UI: background #0f172a, cards #1e293b, borders #334155

## Code Style
- Use const/let, never var
- Always prefer async/await over callbacks
- Return early for validation failures (guard clauses)
- All routes must use try/catch with proper error responses
- All API responses return JSON
- Error responses include { error: "message" }
- HTTP status codes: 200 OK, 201 Created, 400 Bad Request, 404 Not Found
- SQL uses parameterized queries (never string concatenation)

## Git Conventions
- Conventional commits: feat:, fix:, docs:, refactor:
- One logical change per commit
- No committing node_modules or .db files

## File Structure
- src/ for server code
- public/ for frontend files
- Database file: dashboard.db (auto-created on first run)

## Testing
- Test API endpoints after every change
- Verify frontend loads and displays data after changes
```

3. Verify Claude Reads It

Start a new context to pick up the CLAUDE.md:

```
/clear
```

Then ask:

```
What conventions does this project follow?
```

Expected output: Claude recites your rules back to you — code style, response format, Git conventions, file structure. This confirms CLAUDE.md is loaded.

4. Test Convention Enforcement

Now ask Claude to make a change:

```
Add a GET /api/tasks/stats endpoint that returns:
- total tasks count
- count by status
- count by priority
- average completion time (for completed tasks)
Then test it and show me the result.
```

Watch Claude follow your conventions automatically: - Uses `const` (not `var`) — your code style rule - Returns early for edge cases — your guard clause rule - Wraps in try/catch — your error handling rule - Returns proper JSON with error handling — your API rule - Uses parameterized SQL — your security rule

5. Commit

```
Commit this change following our git conventions
```

Claude should use `feat: add task statistics endpoint` or similar.

What Just Happened?

CLAUDE.md is magic. Here's why:

Every time Claude starts working, it automatically: 1. Looks for `CLAUDE.md` in the project root 2. Reads it before doing anything else 3. Treats it as the source of truth for your project

You wrote ONE file with your conventions, and now: - Every endpoint Claude creates follows your response format - Every function uses async/await - Every route has try/catch - File placement matches your structure

Without CLAUDE.md: You'd have to remind Claude every time: "Use async/await. Make sure error handling is consistent. Follow this response format..."

With CLAUDE.md: Claude just knows.

> ***The mental shift:*** *You didn't ask for try/catch. You didn't ask for guard clauses. CLAUDE.md made those automatic. That's the difference between a prompt and a policy.*

Add a Convention and Watch It Take Effect

Tell Claude to add a new convention:

```
Add a Logging section to CLAUDE.md with these rules:
- All routes must log request method, path, and response time to console
- Format: [timestamp] METHOD /path - STATUS (XXXms)
```

Clear context so Claude picks up the change, and test it:

```
/clear
```

```
Add a GET /api/health endpoint that returns the server status and uptime
```

Check the response. Does it include logging? Claude should follow the new convention immediately.

Optional: Go Deeper (if you finish early)

Add a Testing section to CLAUDE.md:

```
Add a Testing section to CLAUDE.md: After creating any new endpoint,
Claude must automatically test it with at least 2 test cases
(one success, one error case) and report the results.
```

Then clear context and add another endpoint. Does Claude test it automatically?

Try conflicting conventions:

```
Add this rule to CLAUDE.md under Code Style:
"All functions must use callback style, never async/await"
```

Clear context, then add an endpoint. What does Claude do when two rules conflict? (The newer rule wins — but Claude may ask for clarification. This teaches students that CLAUDE.md order matters.)

Create a CLAUDE.md for your real project:

Think about your actual team's coding standards. Start drafting a CLAUDE.md you'd actually use at work. What conventions would you include? What rules do you wish were automatic?

□Lab B Checkpoint

- [ ] CLAUDE.md file exists in project root
- [ ] Claude follows your conventions automatically (const, try/catch, guard clauses)
- [ ] Stats endpoint works
- [ ] Logging convention added and working
- [ ] Clean conventional commits

□Return to slides when your instructor is ready.

Lab C: Precision & Recovery (30-45 min)

After: Context Feeding & @ Mentions slides

Goal

Master context control with @ mentions to make Claude look exactly where you want — and learn to recover when things break.

Steps

1. Precision Refactoring with @ Mentions

Start with a broad prompt to see the difference:

```
Update the API to handle errors better.
```

Notice: Claude has to search your entire project, decide which files matter, and guess what "better" means. This burns tokens and produces generic results.

Now try with precision:

```
Look at @src/index.js — the routes are all in one file.
Refactor: extract metric routes into src/routes/metrics.js
and task routes into src/routes/tasks.js.
Keep the main file clean with just app setup, middleware, and route mounting.
```

Watch Claude: 1. Read the specific file you mentioned 2. Identify the route groups 3. Create two new files following your CLAUDE.md conventions 4. Update the main file to import and mount them 5. Test that nothing broke

> ***Why this matters:*** *Every token Claude reads costs money and uses limited context window space. @ mentions load exactly the files you need — nothing more. In a large codebase, the difference between* `@src/auth.ts` *and "look at the auth code" could be 50,000 tokens of unnecessary context.*

2. Verify Nothing Broke

Ask Claude to verify:

```
Test all the API endpoints — metrics, tasks, dashboard, and stats.
Show me the results.
```

Expected output: Claude runs the tests and all endpoints return valid data.

3. Intentional Break & Recovery

This is how developers actually use Claude Code — things break, and you fix them.

Ask Claude to simulate a problem:

```
Delete the file src/routes/metrics.js
```

Now recover:

```
The metrics routes file was deleted. Restore it based on the original
API spec and our CLAUDE.md conventions. Make sure all metric endpoints work.
```

Watch Claude: 1. Reads the remaining code to understand what's missing 2. Checks CLAUDE.md for conventions 3. Rebuilds the file following your rules 4. Tests that it works

What Just Happened?

Recovery prompting is a core skill. Claude's diff awareness means it can reconstruct what's missing by analyzing what's still there.

- Claude read `src/routes/tasks.js` to understand the pattern
- It checked `src/index.js` to see what routes were being imported
- It regenerated the metrics routes following the same conventions

- It verified everything still works

In the real world, you'll use this pattern when: - A merge conflict corrupts a file - You accidentally delete something - A dependency upgrade breaks existing code - You need to reverse-engineer a missing component

> ***The pattern:*** *Describe what's wrong + what the result should look like. Claude handles the detective work.*

---

4. Add a Feature Using Multiple File References

```
Look at @src/routes/tasks.js and @public/index.html — add the ability
to create a new task from the frontend UI. Add a simple form at the top
of the tasks section with fields for title, priority (dropdown), and
assigned_to. When submitted, POST to the API and refresh the task list.
```

5. Test the Form

Open `http://localhost:3100` in your browser. You should see a form above the task list. Create a task and verify it appears.

6. Commit

```
Commit these changes as two separate commits:
one for the refactor, one for the new feature
```

Watch Claude organize changes into logical groups and write descriptive conventional commit messages for each.

Optional: Go Deeper (if you finish early)

URL @ mentions — bring in external docs:

```
@https://expressjs.com/en/guide/error-handling.html Update our error
handling middleware to match Express best practices from this guide
```

Claude will fetch the Express documentation, read the patterns, and update your code.

Multi-file investigation:

```
Look at @src/routes/metrics.js and @src/routes/tasks.js — are there any
patterns that could be extracted into a shared utility? If so, create
src/utils/response.js with shared helpers and refactor both route files to use it.
```

Stress test the recovery:

Delete TWO files at once and see if Claude can recover:

```
Delete src/routes/metrics.js and src/routes/tasks.js
```

Then:

```
Both route files were deleted. Restore them both based on the API spec
in CLAUDE.md and the current state of src/index.js. Test everything.
```

The "explain it to me" pattern:

```
Walk me through @src/index.js line by line.
Explain what each section does and why it's structured this way.
```

This is great for learning codebases you didn't write.

□Lab C Checkpoint

- [ ] Routes split into separate files (src/routes/)
- [ ] Recovery from deleted file successful
- [ ] All API endpoints still work
- [ ] Frontend has a "New Task" form that creates tasks
- [ ] Two clean git commits (refactor + feature)

□Return to slides when your instructor is ready.

Lab D: Ship It (30-45 min)

After: Git Workflow & Best Practices slides

Goal

Polish the dashboard into something professional, then use Git workflows in natural language. Start rough, iterate to great — this is how professionals use Claude Code every day.

Steps

1. Add Data Visualization

```
Add a simple bar chart to the dashboard showing task counts by status
(pending, in-progress, completed). Use vanilla JavaScript and CSS —
no chart libraries. Make it look good with our dark theme colors.
```

2. Verify in Browser

Open `http://localhost:3100` and look at the chart.

3. Critique and Refine

Look at the result, then tell Claude what to improve:

```
The chart works but [describe what you'd change — colors, spacing, labels, etc.]
Make it better.
```

> ***This is the key skill.*** *You don't need to know how to fix it. You just need to know what "better" looks like and describe it. Claude handles the implementation.*

Try 2-3 rounds of refinement. Each time, describe what's not right and let Claude fix it.

4. Add Search and Filter

```
Add a filter bar above the tasks list:
- Dropdown to filter by status (All, Pending, In Progress, Completed)
- Dropdown to filter by priority (All, High, Medium, Low)
- Filters should update the list immediately (no page reload)
```

Test the filters in your browser.

5. Production Hardening

Ask Claude an architectural question:

```
What would you change about this codebase before deploying to production?
```

Review the suggestions, then:

```
Implement the top 3 most important production improvements you just suggested.
```

> ***You just used Claude as an architectural partner**, not just a code generator. The "what would you change?" pattern is incredibly powerful — Claude knows the difference between demo code and production code.*

6. Commit Your Polish Work

```
Commit all improvements with message "feat: add chart, filters, and production
hardening"
```

---

7. Feature Branch Workflow

```
Create a new git branch called "feature/export" and switch to it
```

8. Build the Export Feature

```
Add a CSV export feature:
1. New endpoint: GET /api/export/metrics?format=csv returns metrics as downloadable CSV
2. New endpoint: GET /api/export/tasks?format=csv returns tasks as downloadable CSV
3. Add "Export CSV" buttons to the dashboard UI for both metrics and tasks
4. Buttons should trigger file downloads
```

9. Test the Export

Ask Claude to verify:

```
Test both CSV export endpoints and show me the output
```

Also test the buttons in your browser — they should download .csv files.

10. Commit and Merge

```
Commit the export feature, switch back to main, and merge the feature branch
```

11. Review Your Git History

```
Show me the git log with a nice one-line format
```

Expected output: Clean conventional commits telling the story of your project:

```
abc1234 feat: add CSV export for metrics and tasks
def5678 feat: add chart, filters, and production hardening
ghi9012 feat: add task creation form to frontend
jkl3456 refactor: extract routes into separate files
mno7890 feat: add task statistics endpoint
pqr1234 feat: initial business dashboard with API and UI
```

Optional: Go Deeper (if you finish early)

The "interview me" pattern:

```
I want to add user authentication to this dashboard. Before writing any code,
interview me: ask me 5 questions about my requirements so you build exactly
what I need.
```

Answer Claude's questions, then let it build the auth system. Notice how much better the result is when Claude asks first.

Code review your own project:

```
Review the entire codebase against our CLAUDE.md conventions.
Are there any violations? List them and fix the top 3.
```

UI polish round:

```
Look at @public/index.html — the dashboard needs these improvements:
1. Add a last-updated timestamp that refreshes every 30 seconds
2. Metrics cards should show a colored indicator: green for positive trends, red for is
sues
3. The task list should show how long ago each task was created (e.g., "2 hours ago")
4. Add subtle hover effects on cards and task items
```

Add a completely new feature of YOUR choice:

Think about what would make this dashboard useful for YOUR business. Ask Claude to build it. Examples: - Email notifications when a high-priority task is created - A notes/comments field on each

task - Metric history charts showing trends over time - User roles (admin vs viewer) - Dark/light theme toggle

---

□Lab D Final Checkpoint

Your completed Business Dashboard includes: - [ ] REST API with 10+ endpoints - [ ] SQLite database (zero configuration) - [ ] Professional dark-themed web UI - [ ] Input validation with proper error responses - [ ] Bar chart visualization - [ ] Task filters (status + priority) - [ ] CSV export for metrics and tasks - [ ] Add-task form on the frontend - [ ] Production hardening (env vars, security headers, logging) - [ ] CLAUDE.md with team conventions - [ ] Clean git history with conventional commits - [ ] Feature branch workflow

---

What You Built vs. What You Typed

Count your prompts. You probably typed **fewer than 25 messages** to Claude Code and got:

- ~500 lines of backend code
- ~200 lines of frontend code
- A working SQLite database with sample data
- A professional UI with charts, filters, and forms
- Production-quality error handling and security
- Clean git history with 6+ conventional commits

That's the leverage. You described what you wanted. Claude Code built it.

Key Takeaways

1. The Agentic Loop (Lab A) - Claude doesn't just complete code — it plans, executes, and validates - You provide intent, Claude provides implementation

2. CLAUDE.md is Your Team Brain (Lab B) - One file that teaches Claude your conventions - Automatic adherence to standards — no reminding

3. Context Control with @ Mentions (Lab C) - @ mentions = surgical precision for Claude's attention - Recovery prompting rebuilds what's broken

4. Iterate to Great + Ship with Git (Lab D) - Start rough, polish progressively - "What would you change?" turns Claude into an architect - Git workflows in natural language

---

Continues in Day 3

Tomorrow, you'll take this exact dashboard and add: - Custom slash commands for common operations - Security hooks (secret detection, audit logging) - A report-generation skill - MCP integration for live data - Deployment with subagents working in parallel

Don't delete this project. You'll need it tomorrow.

Troubleshooting

Issue	Fix
Port 3100 in use	Tell Claude: "The port is in use, switch to 3200"
`better-sqlite3` won't install	Tell Claude: "better-sqlite3 failed to install, fix it"
Database locked	Tell Claude: "The database is locked, stop any running servers and restart"
Claude seems confused	Run `/clear` and re-state what you want with @ file references
Git conflicts	Tell Claude: "Resolve any git conflicts and show me the result"
Node version too old	Run `node -v` — must be 18+. Install latest from nodejs.org
Claude Code not found	See Pre-Flight section for installation commands
Slow responses	Try `/model sonnet` for faster (but slightly less capable) responses
Context too large	Run `/compact` to compress conversation history
Claude not following conventions	Verify `CLAUDE.md` is in project root. Try `/clear` so Claude re-reads it
`tmpclaude-*` files in project	Temp files from Claude's editing process. Safe to delete. Add `tmpclaude*` to `.gitignore`